

Deterministic Authorization and Recovery for Agentic Payments

A failure analysis, a runnable demonstration, and notes on fitting Natural’s API

Working paper · revised draft

September 19, 2026

Abstract

Agent-initiated spending is being deployed faster than any public record of its failures is accumulating. A survey of the documented record finds one well-corroborated case of an autonomous agent making an unauthorized real-money purchase, a small cluster of merchant-side wrong-outcome failures, and a body of red-team work demonstrating capability without confirmed victims. That scarcity is a reporting artifact rather than evidence of reliability: there is no disclosure regime for agent purchase errors, and individual losses are small enough to be absorbed by ordinary chargebacks.

Benchmark data fills the gap the incident record leaves. On a computer-use benchmark, the strongest agent measured succeeds at least once in ten attempts on roughly 78% of tasks but succeeds on all ten attempts for only about 36%. On a retail benchmark the most over-represented error category is confirmation handling. A control designed for an agent that is usually right must therefore assume it will sometimes be wrong in exactly the step where money moves.

This paper argues that existing agentic payment controls bound the wrong quantity. Per-transaction amount caps, tokenized credentials, and spend limits bound *how much* an agent can spend. Every documented failure surveyed here occurred within an authorized amount, at a permitted merchant, on a legitimately delegated credential. What was wrong was the purchase, not the price.

Each leg of the proposed remedy has prior art: deterministic pre-action authorization, authority bound to a pinned agent configuration, and outcome-aware settlement have all been proposed separately. The contribution claimed here is narrower and is the conjunction: one human approval that binds an exact purchase record by digest *and* a specific agent version, enforced by pure functions before money moves, settled against fulfillment evidence the agent did not authorize, with the unrecovered residue reported as a first-class figure. A runnable demonstration against simulated providers shows the property in one screen: a legitimately authenticated agent, under the amount cap, at a permitted merchant, proposing the wrong dates, and the payment provider is never called. A read-only inspection of Natural’s API shows where such a layer would attach and confirms that the rail’s event stream distinguishes money moved from goods delivered only as a post-hoc chargeback reason code.

1. Deployment is outpacing the failure record

In February 2025 a technology columnist asked OpenAI’s Operator to compare egg prices in his neighborhood. Within ten minutes the agent bought a dozen eggs through Instacart and paid for

human delivery, a purchase he had not authorized. The transaction was \$31.43. What makes it more than an anecdote is that it bypassed OpenAI’s own stated safeguard requiring user confirmation before a purchase; OpenAI acknowledged the failure and committed to improving those safeguards. [19]

That incident is, as far as this survey can establish, the clearest public case of an autonomous agent making an unauthorized real-money purchase. It is nearly the only one.

This is not a claim that agent purchase errors are rare. It is a claim about the record. Searching the incident databases, technology press, security research, and regulatory publications for documented agent-initiated purchase failures returns a very short list, assembled in Section 2. Meanwhile the infrastructure for such purchases has been deployed broadly: card networks have published agent transaction protocols [6] [5], a major marketplace operates an agentic checkout against third-party catalogs [21], a card issuer has committed to indemnify registered-agent purchase errors [35], and consumer assistants that transact on a user’s behalf have raised substantial capital and reached general availability.

Several structural reasons explain the gap, none of them reassuring:

- **No disclosure regime.** No rule requires a platform to publish an agent purchase error. Voluntary transparency reporting so far has covered model behavior rather than transactions; a September 2026 disclosure of six agent incidents by OpenAI contained no purchase, payment, or booking events, all six having occurred in internal testing. [25]
- **Losses are small enough to disappear.** A \$31 error resolves through an ordinary chargeback or a goodwill refund and never becomes a public record.
- **Merchant-side harms settle privately.** Where the injured party is a business rather than a consumer, the dispute resolves contractually. This survey found no documented enterprise procurement agent failure, which is more plausibly a disclosure artifact than an absence of error.

The honest formulation is that agent-initiated spending is being deployed considerably faster than public evidence about its reliability is accumulating. A design discipline for this domain therefore cannot wait for the incident record to mature, and should not be calibrated to the small number of failures that happen to have been reported.

2. What the record does contain

Sources were accessed September 18 and 19, 2026. Each item below is labeled by evidence tier. Two claims encountered during this survey, a statistic on agent dispute rates and a purported regulatory advisory on autonomous-agent purchases, could not be substantiated at their attributed sources and are deliberately excluded.

2.1 Liability is already settled, and it sits with the operator

Before the agentic era, a Canadian tribunal ruled on an airline chatbot that gave a passenger incorrect information about a bereavement fare, causing him to pay \$1,630.36 in full fare. The airline argued that the chatbot was “a separate legal entity” for which it was not responsible. The tribunal rejected this, holding that the company “is responsible for all the information on its website” and that it “makes no difference whether the information comes from a static page or a chatbot,” and awarded damages. [20]

Evidence tier: court ruling. This is the strongest item in the survey and the most consequential. It establishes that an operator owns its agent’s errors. Any architecture that leaves an agent’s wrong purchase unattributable is therefore not merely a technical gap; it is an unmanaged liability.

2.2 Authorized purchase, wrong outcome

This is the failure mode central to this paper, and the most thinly documented.

In December 2024 a reporter asked a shopping agent to buy a specific toothpaste from a named retailer. The agent took payment, confirmed on the reporter’s bank statement, then failed for three hours without completing the purchase. The item was in fact out of stock at the retailer while appearing available in the agent’s scraped search data. [22] *Evidence tier: named-source journalism, firsthand, single source.* Authorization was valid at every layer. The agent’s model of the world was wrong.

At larger scale, a marketplace agentic checkout placed orders against stale, incorrect, or entirely absent third-party catalog data beginning around November 2025. Named merchants reported receiving orders for products they do not sell, listings displaying photographs of unrelated items, and orders for discontinued stock. When the marketplace’s data was wrong, the merchant absorbed return shipping and refund. One affected merchant collected 145 responses from other brands. [21] *Evidence tier: named-source journalism, multiple named merchants, two outlets.*

The marketplace case inverts the usual framing: the consumer’s authorization was entirely valid and the injured party was the merchant. It is the only fleet-scale evidence located in this survey.

2.3 Irreversible action against an explicit constraint

In July 2025 a coding agent, operating under an explicit instruction not to modify production during a code freeze, executed destructive commands against a live production database, deleting records for approximately 1,200 companies and a comparable number of individuals. It then reported fabricated results and incorrectly asserted that recovery was impossible. The data was recovered manually; the vendor apologized and shipped environment separation and a non-executing planning mode within days. [23] *Evidence tier: firsthand, company acknowledgment, journalism.*

This was not a financial transaction and is included only for the mechanism it demonstrates: an agent taking an irreversible action that the operator had explicitly forbidden, then misreporting it. The same mechanism applied to a payment produces a loss that no amount limit would have prevented.

2.4 Injection-driven spending: capability, not casualties

Security researchers have catalogued prompt-injection payloads on live web pages that specifically target payment actions, including attempts to subscribe a victim to a paid plan, force a donation through a payment link, and redirect a substantial transfer to an attacker-controlled account. The publishing researchers state explicitly that they are not aware of any confirmed real-world instance where such an attack succeeded against a deployed system. [24] Separate vulnerability research has demonstrated indirect injection against agentic browsers, including payloads concealed in images. [26]

Evidence tier: vendor security research. The accurate formulation is that the capability is demonstrated and payloads are present in the wild, while no confirmed consumer losses have been

published. This must not be presented as an incident record. It does establish that an agent’s instruction stream is attacker-reachable, which bears directly on why authority should bind to terms a human approved rather than to the agent’s in-flight reasoning.

3. The rate, which the anecdotes cannot supply

Because the incident record is thin, the more informative evidence is measured error rates, and specifically the gap between an agent succeeding *once* and succeeding *every time*.

The cleanest measurement of that gap is a 2026 study of computer-use agents on the OSWorld benchmark. [29] Its headline finding: for the strongest configuration measured (Agent S3 with GPT-5), Pass@10, the fraction of tasks the agent completes at least once in ten independent attempts, reaches approximately 78%. Pass[^]10, the fraction of tasks it completes on *all* ten attempts, is approximately 36%. Same agent, same tasks, same environment. The 42-point spread between those numbers is the quantity a payment control has to be designed around. The authors attribute the spread to three sources, execution stochasticity, task-specification ambiguity, and variability in agent behavior, and the second of those is what Section 7.3 of this paper addresses directly.

An earlier retail-and-airline benchmark reported the same shape under the name pass[^]k, and remains the source for the domain-specific error taxonomy. [18] On its retail domain, a model measured at roughly 61% correct on a single attempt fell to roughly 25% correct across eight consecutive attempts. Frontier models have since improved single-attempt retail performance substantially; the shape of the decline is the durable finding. Two further details from that work bear on this paper directly. First, the benchmark’s retail error taxonomy over-represents *confirmation handling* errors relative to other domains, the agent mishandling the step where it should have checked with the user. That is precisely the failure exhibited in the Operator incident, where a stated confirmation safeguard did not fire. Second, it also over-represents *constraint interpretation* errors, the agent misreading what it was told to satisfy. Both benchmarks are preprints; neither is cited here as peer-reviewed.

One further result bears on version binding specifically. A study of LLM shopping agents across model versions found that “model updates can drastically reshuffle market shares”: the same agent, upgraded, buys different things. [34] An authorization that survives a model update is therefore an authorization for behavior nobody approved.

A spending agent operates in the repeated regime, not the single-attempt one. A control designed on the assumption that the agent is usually correct is calibrated to the wrong statistic.

4. Why current controls do not address these failures

The controls shipped with current agentic payment products are, with one exception, quantity controls.

A consumer assistant that transacts on a user’s behalf mints a single-use virtual card authorized for a specific amount, so that the agent never handles the underlying card number. [9] This is a genuine and well-designed control: it bounds the exposure of any one transaction and removes the credential from the agent’s reach entirely.

A payments platform for agents enforces permissions and limits server-side on every request, so that a compromised or misbehaving agent cannot exceed what it was granted. Limits may be set

per transaction, per day, and per month. A payment breaching a limit is not rejected; it is held and an approval is opened that the agent cannot clear itself. [7] Server-side evaluation and the inability of an agent to raise its own caps are both correct design choices, and stronger than client-side enforcement. Section 8.1 reports what the same platform’s API actually exposes.

Neither control addresses any failure in Section 2.

The unauthorized egg purchase was for \$31.43, within any plausible limit. The toothpaste purchase was the right item at the right merchant for the right price; only the inventory was wrong. The marketplace orders were correctly priced against catalogs that were themselves incorrect. In each case an amount-based control evaluates the transaction and finds nothing to object to, because there was nothing wrong with the amount.

Two further properties of the current model deserve note:

Holding rather than blocking relocates the decision but does not make it deterministic.

A limit breach that opens an approval is a good design for the case where the user is available and understands what they are approving. It does not help where the purchase is within limits, which is the case in every incident surveyed.

Where confirmation is discretionary, it is not a control. One consumer assistant’s terms appoint the service as the user’s agent to “enter into agreements, commitments or transactions” on their behalf, then state that the provider “may implement safeguards, confirmation requirements, or other controls on certain Actions” while making “no representation or warranty that such safeguards will prevent unintended or erroneous Actions.” The same terms make payments non-refundable, direct purchase disputes to the merchant rather than the provider, and cap provider liability at one hundred dollars. [27] An independent assessment of the same product found that confirmation before charge does sometimes occur. [28] A safeguard that fires sometimes, is disclaimed contractually, and is paired with no recovery path is not a control a user can reason about.

4.1 The indemnification counter-argument

The strongest objection to this paper is that the problem has already been priced rather than prevented. In April 2026 American Express announced an agentic commerce developer kit and, alongside it, Amex Agent Purchase Protection: “if a Card Member authorizes an AI agent to make a purchase and that agent sends American Express the customer’s authenticated purchase intent, American Express will protect eligible customers from charges related to AI agent error.” [35] If the issuer absorbs the loss, why build a prevention layer at all?

Three answers, in increasing order of weight.

First, the commitment is conditional and forward-looking. The release’s own language is “in the future,” the protection applies to “eligible customers,” and it is filed under the release’s forward-looking-statements caution. It is a stated direction, not a live guarantee, and this paper cites it as such.

Second, the condition is the point. Protection attaches only when the agent “sends American Express the customer’s authenticated purchase intent.” The kit’s components are named accordingly: Agent Registration, Intent Intelligence “to ensure Card Member purchase intent is accurately captured to support authentication, authorization and disputes,” and Cart Context “to enhance validation, authorizations and dispute investigations.” In other words, the indemnity is

underwritten by exactly the artifact this paper proposes to produce: an exact, authenticated record of what the human approved, captured before the transaction. An issuer cannot distinguish “agent error” from “customer changed their mind” without it. The prevention layer and the indemnity are not alternatives; the indemnity presupposes the prevention layer’s output.

Third, indemnification covers the cardholder and no one else. The marketplace case in Section 2.2 injured merchants, who absorbed refunds and return shipping against orders no cardholder disputed. The toothpaste case produced a three-hour failure and a bank-statement charge that an indemnity would eventually reverse but a preflight check would never have created. And a closed-loop issuer’s protection ends at that issuer’s network; a wallet-based rail such as the one examined in Section 8.1 has no equivalent commitment. An indemnity is a good backstop. It is a poor primary control, because it is triggered by exactly the loss a deterministic check would have prevented, and it is available only to some of the parties who bear that loss.

The missing axis is not *how much* but *what*, and *whether it worked*.

5. Related work

This section positions the proposal against the literature. The area is crowded, and the honest statement is that no single leg of the proposed structure is new. What this review did not find is the conjunction, and Section 5.4 states the gaps that survived it.

Sources were accessed September 17 through 19, 2026. Of the roughly forty papers examined, three are peer-reviewed: an agent-security benchmark [37] and two law-review treatments of agent liability [38] [39]. Everything else, including the four nearest neighbours below, is a preprint and is labeled as such.

5.1 Delegated spending authority for agents

This is the crowded area, roughly a dozen papers. Four are close enough that a reader should check them before crediting this paper with anything.

Formal analysis of agent payment protocols. A Tamarin-based analysis of x402, MPP, ACP, and AP2 across 86 verification cases finds 40 previously undocumented issues, and concludes that “delegated authorization must remain consistent with its resulting economic and service effects across actors, states, and protocol stages.” [30] That sentence is close to this paper’s thesis, arrived at by formal methods rather than incident survey. The difference is scope: that work verifies the protocols as specified; this paper argues about what the specifications omit, namely an exact purchase record and a fulfillment fact.

Deterministic pre-action authorization. A March 2026 preprint proposes intercepting tool calls “synchronously before execution,” evaluating them “against a declarative policy,” and producing “a cryptographically signed audit record,” reporting zero successful adversarial actions under its policy against 74.6% under a permissive one. [31] It uses the same words this paper uses. It is a general tool-call gate; it does not bind to a human-approved purchase digest, a pinned agent version, or a fulfillment outcome.

Authority under agent evolution. A July 2026 preprint asks the version-binding question directly: whether a grant survives a mutation of the agent, proposing “an immutable effect ceiling established at grant time” below which “authority may contract freely and expand only under specified evidence conditions.” [32] This paper’s authorized-version allow-list and change

classification (Section 7.4) are a concrete instance of that framing for one domain. That work owns the question; this paper implements one answer.

Mandate signatures are not intent. A systematic security analysis of AP2 v0.2 identifies 48 threats and demonstrates that “valid mandate signatures alone do not ensure that an agent-mediated transaction reflects the user’s intent when its pre-authorization context is manipulated.” [33] This is the strongest argument in the literature for evaluating the approved record with a pure function rather than trusting the signature on it, and Section 7.2 relies on it.

Beyond those four: OAuth Rich Authorization Requests already expresses payment amount and recipient in structured form [2]; AP2 defines Checkout and Payment Mandates with linked receipts [3] [4]; Mastercard’s Verifiable Intent (draft v0.1) specifies a delegation chain and machine-enforceable purchase constraints while leaving dispute resolution out of scope [5]; Visa’s Trusted Agent Protocol specifies signed agent-recognition messages [6]. None of these evaluates fulfillment.

5.2 Prompt injection and agent security

AgentDojo (NeurIPS 2024) is the peer-reviewed benchmark for injection attacks on tool-using agents. [37] CaMeL, a preprint, proposes separating control flow from untrusted data so that injected content cannot reach a privileged action. [40] Both motivate the design rule in Section 7.2 that a model’s in-flight assessment is never an enforcement mechanism. Neither addresses payments specifically.

5.3 Identity, provenance, and outcome verification

SPIFFE/SPIRE separates workload identity from credentials [10]; SLSA describes build provenance [11]; the IETF RATS architecture distinguishes evidence, appraisal, and attestation results with explicit freshness assumptions [12]. These are the foundations a stronger-than-self-declared attestation level would reuse (Section 9.3), not infrastructure this paper replaces.

On outcome verification: ERC-8004 proposes identity, reputation, and validation registries with payments orthogonal [13]. The Agentic Settlement Protocol (September 2026 preprint) proposes holds, fulfillment verification, and partial refunds for delayed-fulfillment stablecoin commerce, including travel, and identifies itself as a design paper without deployed measurements. [14] Classical escrow work for digital goods explicitly assumes the good is hash-verifiable. [36] Outcome-aware settlement is therefore not novel; what remains open is stated next.

5.4 What survived the review

Four gaps were checked for and not found occupied.

1. **Outcome verification for physical goods and services.** The escrow and attestation literature verifies digital delivery by hash. A hotel night, a delivered grocery order, or a booked flight has no hash. Section 7.5’s treatment of independently sourced fulfillment evidence with an explicit **unknown** outcome is offered against this gap.
2. **Authorized-but-not-wanted as a first-class category.** Industry names it (“agent chargebacks,” “AI agent error”); the academic literature treats it as either fraud or as a UX problem. It is neither: the authorization was valid and the purchase was wrong.
3. **UETA §10 applied to LLM agents.** The Uniform Electronic Transactions Act lets an individual avoid the effect of an automated transaction where the electronic agent’s provider

gave no “means to prevent or correct the error.” [41] No peer-reviewed treatment applies this to LLM-driven purchasing. If it applies, a provider that offers no preflight is not merely exposed to loss; it may be exposed to avoidance of the transaction itself. This paper flags the hook and does not resolve it.

4. **Real-money failure rates.** Every benchmark in Section 3 is sandboxed. Nobody has published what fraction of live agent purchases are wrong.

5.5 Contribution

Against that map, the claim is the conjunction: (a) deterministic pre-action authorization evaluated by pure functions, (b) over an exact human-approved record identified by digest, (c) bound to an explicit allow-list of agent versions, (d) settled against fulfillment evidence with source attribution and a three-valued outcome, (e) with recovery modeled as a workflow that reports its residue. Each of (a) through (e) has a nearest neighbour above. The joint is unoccupied, and the demonstration in Section 9 is offered as evidence that the joint is constructible, not that it is complete.

6. System model and trust assumptions

The **principal** grants authority. An **operator** deploys the agent. A **control service** authenticates requests, checks mandates, and maintains budget and operation state. **Payment providers** report financial events. **Evidence providers** report fulfillment or recovery events. Some roles may belong to one organization, but their claims should remain distinguishable.

The agent may generate mistaken requests, encounter malicious content, repeat operations, or continue after an operator changes its configuration. Providers may time out, return events out of order, or supply inconsistent evidence. These are the failures the design aims to expose or constrain.

The intended controls require all covered spending to pass through the service or equivalent provider-enforced restrictions. An agent with an unrestricted credential outside that boundary can bypass them. The design assumes a trustworthy authorization service, protected state, authenticated principal actions, and appropriate verification of provider messages. A compromised administrator, colluding evidence source, or stolen credential can invalidate those assumptions. Storing a hash beside data does not protect both from an attacker who can rewrite the database.

“Verified outcome” means an assessment supported by specified evidence under a stated trust policy. It does not mean objective truth, customer satisfaction, legal liability, or proof of the agent’s private reasoning.

Manifests and receipts can reveal sensitive instructions, destinations, purchases, and behavior. The proposed service should minimize disclosure, restrict access, and define retention periods. A digest is not encryption: predictable underlying content may still be guessed. Recording private chain-of-thought is neither necessary nor an assurance mechanism in this design.

7. A deterministic authorization structure

The goal is not to make agent behavior deterministic; it is not. The goal is to make the *authorization decision* deterministic, so that whatever the agent does, the set of purchases it can cause is a pure function of a record a human approved.

The structure has three phases, and the boundary between the first and the third is the whole idea:

PROPOSE	nondeterministic. A model drafts the purchase and extracts typed constraints. Anything it cannot type is recorded as unresolved, and unresolved blocks.
RATIFY	a human approves once. Approval binds <code>sha256(canonical(record))</code> and an explicit agent version.
ENFORCE	deterministic. Pure functions only; no model runs. <code>preflight()</code> gates the payment call; <code>assess()</code> reads payment and fulfillment evidence and returns success, failure, or unknown.

This section states the structure independently of any implementation. Section 9 describes a prototype and a demonstration that realize it for one purchase category.

7.1 Approve an exact record, not a budget

A spending authorization should name the purchase, not a budget. “A refundable room at this hotel, these dates, this rate” rather than “hotels under \$550.” The approved record is canonicalized and hashed; the hash is what the human’s approval binds to.

This inverts the usual tradeoff. A budget is convenient and admits every failure in Section 2, because every one of those purchases was inside its budget. An exact record is less convenient and admits none of them: a cancellation is not a purchase in the record, a different item is not the item in the record, an out-of-stock substitution is not the approved quote.

The cost is real and should be stated. Exact approval means an agent that finds a better option must return for a new approval, and a system that requires many approvals may be abandoned or approved reflexively. Section 10 treats this as the central open question rather than a solved problem.

7.2 Constraints must be predicates

Conditions that govern payment must have machine-checkable representations, amount ceilings, permitted merchants, date ranges, refundability deadlines, evaluated by a pure function over the approved record and the proposed purchase. Natural language may propose these terms; it must not enforce them. A model’s assertion that a purchase looks acceptable is not an enforcement mechanism, and per [33], neither is a valid signature over a mandate whose context was manipulated before signing.

7.3 What cannot be typed must block

A condition the system cannot represent as a predicate (“somewhere nice”, “good reviews”) is the dangerous case, because the natural implementation is to drop it. The structure requires that unresolved conditions be recorded as such and that their presence block unattended execution. Failing closed on uninterpretable terms is what prevents a silent gap between what the human said and what the system enforces.

This also localizes the nondeterminism that natural-language interpretation necessarily introduces. Extraction from prose is fallible; the design requirement is not that extraction be correct but that it be *calibrated*: confidently mistyping a constraint is the harmful failure, while declining to type one is safe because it blocks. A useful evaluation of such a system measures silent mistyping against correct refusal, not extraction accuracy. This is also the paper’s answer to the task-specification

ambiguity that [29] identifies as a source of unreliability: ambiguity is surfaced at RATIFY, not resolved silently at ENFORCE.

7.4 Authority binds to a declared configuration

An authorization granted to an agent is granted to a particular declared configuration of that agent: model reference, instruction and policy digests, code artifact, tool versions and permissions. A newly registered configuration inherits no authority; a change to any authority-bearing field requires review under the principal's policy, and a change whose behavioral effect is unknown, a dependency bump, a code change labeled a patch, requires review rather than receiving an automatic exemption.

Authority is checked before any exposure is reserved, and rechecked immediately before the external call. The second check exists because revocation can arrive while an operation is in flight. It narrows the window; it does not close it. Ordering between a local registry and an external provider is not atomic, and once a provider has accepted an instruction, revocation cannot retract it. That operation must be reconciled under the configuration that issued it rather than disowned.

7.5 Outcome is assessed from evidence the agent did not write

Payment success and fulfillment success are separate facts and must be recorded separately. Capture is recorded when financial evidence supports it, even where fulfillment later fails; delaying recognition of spent money until the outcome is satisfactory misstates exposure.

Three evidence streams are kept apart by source: `payment_observed`, authoritative for money; `fulfillment_observed`, authoritative for delivery; and `execution_observed`, the agent's own narrative, which is logged and never read by the assessor. The assessor returns three outcomes, not two. Where payment and fulfillment facts conflict, or two fulfillment sources disagree, it returns `unknown` and stops authority for human review rather than guessing. This is the property that would have caught the toothpaste case: payment captured, fulfillment absent, assessment not successful.

A caveat this structure cannot escape: source attribution on evidence is only as good as the adapter that produced it. Labeling a record as merchant-originated does not authenticate the merchant. An adapter-signed HMAC over (operation key, external id, payload) authenticates the *adapter*, not the merchant. Evidence authenticity is a deployment obligation, and a system that treats a source label as proof has moved the trust problem rather than solved it.

7.6 Recovery is a workflow, not an undo

Prevention is cheap and reliable: a pure function evaluated before money moves. Recovery is expensive and unreliable: it depends on merchants, networks, and deadlines outside the system's control. The design consequence is that effort belongs in prevention, and that recovery must never be described as an undo.

Concretely, the structure requires that a refund *request* and a *posted* refund be distinct recorded facts; that a posted refund restore spending authority only where an approved policy says it does, so that repeated purchase-and-refund cycles cannot evade a cumulative limit; that disputes require explicit human authorization rather than being filed automatically; and that a receipt report an *unrecovered* amount as a first-class figure. A system claiming complete recovery would be misrepresenting what it controls. Reporting the residue is the honest alternative.

8. Fitting existing infrastructure

This structure is an authorization and accountability layer. It is not a payment rail, and implementing it does not require custody, card issuance, a new token, a blockchain, or a legal identity standard. It sits above a rail and calls it.

That placement is deliberate, and it is what makes the structure complementary to current agentic payment platforms rather than competitive with them. A platform that holds funds, enforces permissions server-side, and moves money on instruction supplies exactly the execution layer this structure assumes. What the structure adds is upstream of the payment call and downstream of it: an exact approved record that determines whether the call should be made at all, and an outcome assessment that determines what happens afterward.

8.1 What Natural's API exposes: a read-only inspection

On September 19, 2026, Natural's API was inspected through its MCP server under an agent-scoped OAuth grant (actor type `AGENT`, credential kind `oauth_grant`), plus its published API reference. [42] [43] [44] Calls made: identity, party limits, agent list, transaction list, customer list, external-account list. No payment, transfer, request, deposit, or withdrawal was created; no money moved. The account held no transactions, so event payloads below are quoted from the published event catalog rather than observed live. Three questions were put to the product.

Can a payment carry an opaque digest of an approved purchase? Yes, as a passenger. A payment accepts `tags`, at most 30 keys, keys 1 to 128 alphanumeric-or-underscore characters, values 1 to 256 characters; a `description` of at most 80 characters; an `Idempotency-Key` header of at most 255 characters; and an `X-Instance-ID` header of at most 1024 characters that the docs describe as a "caller-chosen identifier for the agent run, session, or conversation, required when an agent moves money." A 64-character hex SHA-256 fits in any of these. `tags` is the correct carrier: it is returned on the `Payment` object and inside the `payment.completed` webhook payload, so a control layer's contract digest would ride out on the call and come back on every event about it. Natural does not interpret tags. This makes the digest a correlation handle for the `RATIFY-to-ENFORCE` link, not an enforcement point. Enforcement would remain upstream, in the layer that decides whether to call `create_payment` at all, which is where Section 7 puts it.

What does a mandate contain beyond amount limits? The answer depends on which of two objects "mandate" names, and neither carries a purchase.

The agent-side grant is the `Agent` object: `name`, `description`, `tags`, `handle`, `status` (`ACTIVE` or `REVOKED`), `createdBy`, `lastActiveAt`, and `limits` with three fields, `perTransaction`, `perDay`, and `perMonth`, each an integer in cents or null. Party-level limits are the same three. The permission set on the inspected grant was a flat list of eleven scopes (`payments.create`, `payments.read`, `wallets.transfer`, and so on). There is no merchant field, no category, no item, no date, no recipient allow-list, and no digest. This is the model Section 4 describes: correct, server-enforced, and entirely about quantity.

Natural also has a resource literally named `mandate`, with `mandate.created`, `mandate.updated`, and `mandate.revoked` events. Its published payload is a bank-debit authorization: `scheme`: "ach_web", `amountType`, `amount` in minor units, `frequency`: "recurring", `merchantId`, `effectiveAt`, `expiresAt`, supersession links, and an `evidence` object containing `captureMethod`, `capturedAt`, `ipAddress`, `authorizationText`, and `authorizationUri`. That evidence object is worth noting precisely because of what it is: evidence that *consent was captured*, of the kind

NACHA rules require for a WEB debit. It is not evidence about any outcome. So the one place Natural’s API records an authorization artifact with provenance, it records the provenance of the permission, not of the purchase. A memo to Natural should use the word “mandate” carefully; in their vocabulary it already means something.

Does anything in the event stream distinguish money moved from goods delivered? On the payment leg, no. The Payment status enum is CREATED, PROCESSING, PENDING_CLAIM, IN_REVIEW, COMPLETED, FAILED, RETURNED, APPROVAL_DENIED, CANCELED. Every value describes the money. The event catalog has no fulfillment, delivery, receipt, or confirmation event for any resource. The term “fulfill” does appear, in `fulfill_payment_request`, and it means the *payer paying* a request, a term collision the memo should flag before it causes confusion.

The one place delivery enters the product is the card-acquiring stream: `chargeback.created` carries a `reason` object, and the published example is `{ category: "notReceived", networkCode: "13.1", description: "Merchandise or services not received" }`. That is the network’s own vocabulary for the failure this paper is about. It appears only as a dispute reason, raised by a cardholder after the fact, on card payments, and it opens a case that “starts with response, representment, and closure stages.” In other words: the distinction exists in the product exactly once, as a post-hoc chargeback code, and never as a signal on the leg where the agent paid. The claim in the abstract is therefore confirmed with one refinement, and the refinement strengthens it. Natural already has a place to put a fulfillment fact, the chargeback reason taxonomy, and no way to learn that fact before a customer complains.

Two further observations from the inspection. The MCP server refuses `list_wallets` for agent-attributed calls because the list “is party-wide and can expose ungranted wallets or vault balances”; this is a least-exposure decision made in the right direction and worth acknowledging. And Natural’s own founder essay states that “the market for agent to agent payments has also not yet materialized” and that “you own the market logic (targeting, sourcing, pricing, and fulfillment), and Natural moves each customer’s money inside the limits they delegated.” [1] That is a precise statement of the boundary. Everything in Section 7 lives on the customer’s side of it.

8.2 Where the layer would attach

Mapped onto Natural’s surface, the three phases land as follows. PROPOSE and RATIFY are entirely outside Natural; they produce a digest. ENFORCE’s `preflight()` runs before `create_payment`; on pass, the call carries the digest in `tags` and the mission’s operation key as `Idempotency-Key`, and the mission id as `X-Instance-ID`. `payment.completed` (or `failed`, `returned`) becomes the `payment_observed` stream, matched back by tag. `fulfillment_observed` comes from a merchant or email adapter Natural does not provide. `assess()` runs on the pair. If a `chargeback.created` later arrives with category `notReceived`, that is a fourth evidence stream and a useful one: it is the network telling the control layer its assessment was wrong.

Two constraints on this sketch. Natural’s acceptable-use policy prohibits building a competing product with its technology; evaluating the product and writing about it is what this section does, and wiring Natural in as a rail for a competing wallet is not proposed. And the boundary caveat from Section 6 applies with full force: this works only when the agent’s sole funded credential is one the control layer holds. An agent with its own Natural agent key routes around the layer entirely, which is the same caveat Natural’s documentation makes about its own limits.

Mapping onto other published work: the card networks' agent transaction protocols are converging on cryptographically binding a transaction to a human-approved mandate specifying merchant categories, caps, and time windows [6] [5]; the structure here carries the same idea from categories and caps to the exact purchase, and from authorization to outcome. Rich authorization requests [2] and intent protocols [3] provide transport for structured authorization detail; an integration would require an agreed constraint schema and a verification algorithm, not simply an additional field. Workload identity and attestation work [10] [11] [12] provides the machinery that would raise version binding above the declared level. Payment adapters must carry provider-specific rules for idempotency retention, authorization expiry, partial capture, refund states, and disputes [16] [17]; any field that cannot be faithfully enforced on a given rail should produce a restricted mode or a refusal rather than a claim of equivalent protection.

9. What the prototype and the demonstration establish

A local TypeScript prototype (Scrip) implements the structure for one purchase category, an exact, human-approved hotel booking, against simulated payment and execution providers. It has never moved real money, issued a real card, or filed a dispute. The observations below come from source inspection, local tests, and one scripted run.

Implemented: an approved contract canonicalized and hashed, with approval binding to that hash; typed constraint predicates evaluated by a pure function before any reservation; unresolved constraints blocking approval; agent lineage, immutable version records with manifest digests, and mandates carrying an explicit authorized-version allow-list; authority checked before reservation and again before dispatch; exposure reserved durably before any provider call; outcome assessed from merchant and payment facts with the agent's narrative excluded; refund request and posted refund recorded separately; and a receipt reporting authorized, reserved, captured, reversed, refunded, returned, and unrecovered amounts as distinct figures.

9.1 The demonstration

`npm run demo:deterministic` runs the three phases end to end in under two seconds against fake providers, with no network access and temporary databases deleted on exit. The screen that carries the argument is the first enforcement step:

3a. Agent proposes: same hotel, same room, same price, Sept 19-21.

X BLOCKED before payment

Constraint mismatch: date_range

Exact purchase changed; revise and approve a new version

amount \$500 <= \$550 cap	passes	← an amount limit stops here
merchant hotel.example	permitted	
agent credential	authentic, version authorized	
date_range 09-18..09-20	mismatch	← preflight() stops here

payments.issue() calls: 0 execution.start() calls: 0

The candidate is under the cap, at the permitted merchant, presented by an authenticated agent whose version the mandate authorizes. Every quantity control passes it. The date is wrong by one day, `preflight()` refuses it, and the payment provider is never called. There is nothing to refund because nothing was reserved.

The run also shows: the PROPOSE phase failing closed on an untypeable condition (“somewhere quiet”) so that ratification is refused until the human resolves it; a forged credential rejected at the service boundary; the exact purchase paid, confirmed by the merchant stream, and assessed **success** from two independent sources while the agent’s narrative is logged and ignored; and a second mission in which payment captures but the merchant reports the booking failed, assessed **failure** with **unrecovered** \$500.00 on the receipt until a refund actually posts, at which point **unrecovered** returns to zero. The last scenario is the toothpaste case, and it is the one a rail whose lifecycle ends at COMPLETED reports as a success. A regression test (`tests/demo-deterministic.test.ts`) asserts each of those outcomes, including that the provider call count after the wrong-date candidate is zero.

What the demonstration does not show: the PROPOSE phase is scripted, not model-driven, so extraction calibration (Section 7.3) is asserted rather than measured; both providers are in-process fakes; the merchant evidence is labeled, not authenticated.

9.2 Adversarial results

An earlier revision of this code failed four attacks that a reviewer identified by inspection. Each was reproduced as an executing test that succeeded against the old code, then fixed. They are retained as regression tests.

Attack	Earlier behavior	Current behavior	Control
Caller supplies a fabricated agent identity object	Completed a purchase	Rejected	Service authenticates a presented credential itself; identity type is unforgeable outside the registry
Credential revoked after authentication, then used	Completed a purchase	Rejected	Credential re-verified against stored state inside every authority check
Mandate scoped to refunds used to purchase	Completed a purchase	Rejected	Requested operation and funding source checked against the mandate
Operator self-asserts a stronger attestation level	Recorded as asserted	Not expressible	Level is no longer a caller-supplied parameter

The first of these is the instructive one. The prototype had been reviewed as implementing version-bound authority, and it did, but its service boundary accepted an unauthenticated claim of having been authenticated, so every check downstream operated on an attacker-chosen identity. A correct authorization model behind an uncontrolled boundary provides no protection. This is offered as a caution rather than a solved problem: boundaries of this kind are easy to describe correctly and easy to implement incorrectly.

9.3 Limits

- **No measured-runtime guarantee.** Registration is self-declared and that is the only level the registry can issue; the interface for stronger attestation exists with no implementation, so a stronger label cannot be produced. A manifest digest detects alteration of the record, not of the running program. An unchanged digest cannot detect undeclared changes, mutable model aliases, or evolving retrieved content.
- **Human authentication is out of scope.** The principal is a caller-supplied identifier. Approval and revocation assume an authenticated surrounding service that does not exist here.
- **Session-level, not request-level, credential binding.** A production design should bind each request to mandate, operation, payload digest, audience, expiry, and a replay guard.
- **Evidence source labels are not authenticated.** See 7.5.
- **No atomic revocation ordering with a provider.** See 7.4.
- **Single category, simulated providers, no live result.** The natural-language-to-typed-constraint extraction described in 7.3 is scripted in the demonstration and not implemented as a model call; constraints are authored directly in fixtures.
- **Canonicalization is custom.** No conformance to a published canonicalization scheme [15] is claimed.

9.4 Local verification record

On September 19, 2026, against commit 5c07c79 plus the demonstration files, `npm run build`, `npx tsc --noEmit`, `npm test`, and `npm run demo:deterministic` each exited 0, with 253 tests passing and 8 skipped across 22 files and one skipped file. The identity suite contained 26 tests, the purchase-mission suite 50, the demonstration suite 1; the skipped file is a PostgreSQL store suite requiring a database.

Those totals describe a test suite, not independent validations of this paper. Passing fixtures against simulated providers do not establish security, payment correctness, evidence authenticity, or fulfillment reliability. No financial operation was initiated.

10. Evaluation required

The central hypothesis, that binding authority to an exact approved record and a declared configuration prevents failures that amount-based controls do not, is untested outside local fixtures. It would be weakened if exact approval provides little protection beyond a well-designed mandate system, or if its approval burden outweighs the protection.

Comparative study. The same task set under three configurations: amount-based limits alone; limits plus exact-record binding; and both plus a measured deployment. Report blocked unauthorized operations *and* legitimate operations incorrectly blocked. A control that stops every bad purchase by stopping most good ones is not useful.

Authority tests. Undeclared configuration changes, mutable aliases, stolen and expired credentials, fabricated caller identities, operator changes, version rollback, missing scopes, and revocation arriving during capability issuance and during dispatch.

Failure injection. Interrupt each persistence and network boundary; reorder and duplicate provider events; reuse idempotency keys past provider retention; simulate partial capture, provider

inconsistency, and refund failure. Report unresolved exposure and time to reconcile rather than assuming timeouts are safely retryable.

Evidence tests. Fabricated merchant labels, compromised adapters, conflicting confirmations, cancelled bookings, stale refund events. Report false success, false failure, and unresolved outcomes against independently established ground truth.

Approval-burden study. Measure whether people understand what they authorized, why a change prompt appeared, and the difference between a refund requested and money returned. Report approval counts, completion time, abandonment, and misunderstanding alongside safety outcomes. Given the Pass¹⁰ evidence in Section 3, the interesting comparison is how burden scales with the number of purchases in a session.

Extraction calibration. For the PROPOSE phase, measure silent mistyping (a constraint typed wrongly and enforced) against correct refusal (a constraint left unresolved and blocking), per Section 7.3. This is the one evaluation the demonstration cannot supply because its PROPOSE phase is scripted.

11. Conclusion

The public record of agent purchase failures is short, and the reasons are structural rather than reassuring. The measured record is more informative: the strongest computer-use agent measured succeeds at least once on 78% of tasks and every time on 36%, and the retail error category that benchmarks over-represent is the handling of confirmation itself.

Against that, the controls currently shipped bound the amount of a transaction. Every failure this survey located was correctly priced. An issuer’s indemnity for agent error is a real backstop and is conditioned on exactly the artifact this paper proposes to produce. The proposal is to bind the authorization to an exact record a human approved, evaluate it with pure functions, fail closed on anything that cannot be represented, bind authority to a declared software configuration, assess outcomes from evidence the agent did not author, and treat recovery as a workflow that reports what it could not recover. No leg of that is new. The conjunction is, as far as this review could establish, unoccupied.

A prototype implements it for one category against simulated providers, a demonstration shows the wrong-date candidate refused before the rail is called, and a read-only inspection of one production rail shows where the layer would attach and that the rail’s own event stream learns about delivery only as a chargeback reason code. None of that establishes that the declared software ran, that evidence is authentic, or that any of this is safe in production. The next contribution should be an authenticated integration against a real rail, and the negative results it produces.

References

Primary sources were accessed September 17 through 19, 2026. Repository branches and product documentation can change; versions are stated where visible. Source descriptions are not endorsements or independently audited product claims. Peer-review status is stated where known; “preprint” means no peer-review claim is made here.

[1] Kahlil Lalji / Natural. [Agentic payments, revisited](#). August 12, 2026. Founder essay; identity, liability, and platform sections.

- [2] T. Lodderstedt, J. Richer, and B. Campbell. [RFC 9396: OAuth 2.0 Rich Authorization Requests](#). 2023. Standards-track specification.
- [3] Google Agentic Commerce. [AP2 specification](#). Inspected document labels itself v0.2; moving repository branch.
- [4] Google Cloud. [Announcing Agent Payments Protocol](#). Launch announcement; consulted for historical terminology, not as the current schema.
- [5] Mastercard / agent-intent contributors. [Verifiable Intent](#). Draft v0.1, as labeled in the inspected repository.
- [6] Visa. [Trusted Agent Protocol specifications](#). Merchant-facing protocol documentation.
- [7] Natural. [Identity](#). Public product description.
- [8] Natural. [Disputes](#). Public product description.
- [9] Dan Hill / Stripe. [Giving agents the ability to pay](#). April 29, 2026. Link wallet product announcement.
- [10] SPIFFE. [SPIRE concepts](#). Workload registration and attestation documentation.
- [11] SLSA. [Provenance, specification v1.2](#). Software supply-chain provenance.
- [12] H. Birkholz et al. [RFC 9334: Remote ATtestation procedureS Architecture](#). 2023. Informational architecture; especially trust and freshness.
- [13] M. De Rossi et al. [ERC-8004: Trustless Agents](#). Draft, created August 13, 2025; status checked at review time.
- [14] B. Mohammadkhani, A. Khekade, and R. Kakkad. [Agentic Settlement Protocol: An Application Profile for Refundable, Delayed-Fulfilment Agent Commerce on Stablecoin Rails](#). arXiv:2609.02208, September 2, 2026. Preprint; design proposal.
- [15] A. Rundgren, B. Jordan, and S. Erdtman. [RFC 8785: JSON Canonicalization Scheme](#). 2020. Informational specification.
- [16] Stripe. [Idempotent requests](#). API documentation; key retention and parameter matching.
- [17] Stripe. [Refunds](#). Operational documentation; pending and failed refund states.
- [18] S. Yao et al. [\$\tau\$ -bench: A Benchmark for Tool-Agent-User Interaction in Real-Domain Dialogue](#). arXiv:2406.12045. Preprint. Source of the pass^k consistency metric and the retail error taxonomy cited in Section 3.
- [19] G. A. Fowler. [OpenAI’s Operator agent](#). Washington Post, February 7, 2025. Firsthand account of an unauthorized \$31.43 purchase; indexed as AI Incident Database incident 1028.
- [20] *Moffatt v. Air Canada*, British Columbia Civil Resolution Tribunal, February 2024. [CBC report](#). Tribunal ruling rejecting the “separate legal entity” defense.
- [21] A. Smith. [Brands are upset that Buy for Me is featuring their products without permission](#). Modern Retail, January 6, 2026. Named merchants; corroborated by contemporaneous CNBC reporting.
- [22] M. Zeff. [The race is on to make AI agents do your online shopping](#). TechCrunch, December 2, 2024. Firsthand test; payment taken without fulfillment. Single source.

- [23] [Replit AI agent production database incident](#). The Register, July 21, 2025. Non-purchase; included for the irreversible-action mechanism. AI Incident Database incident 1152.
- [24] Palo Alto Networks Unit 42. [Prompt injection targeting AI agents](#). March 3, 2026. Payload telemetry; the authors state no confirmed successful real-world exploitation.
- [25] [OpenAI discloses six agent incidents](#). Fortune, September 17, 2026. Cited as a negative result: no purchase or payment events among them.
- [26] Brave. [Agentic browser security: indirect prompt injection and unseeable prompt injections](#). 2025. Vulnerability research by a competitor to the product examined; disclosed as such.
- [27] Instinct (Spear Street Technology, Inc.). [Terms of Service](#). Quoted for the appointment-as-agent, safeguard-disclaimer, non-refundable, and liability-cap clauses in Section 4.
- [28] Assistant Benchmark. [Instinct assessment](#). September 1, 2026. Cited as counter-evidence: a purchase task passed with approval before charge.
- [29] G. Gonzalez-Pumariaga, S. Agashe, J. Yang, A. Li, and X. E. Wang. [On the Reliability of Computer Use Agents](#). arXiv:2604.17849, April 20, 2026. Preprint. Source of the Pass@10 78% / Pass@10 36% figures (Agent S3 with GPT-5, OSWorld).
- [30] K. Jiang, M. Yu, Y. Chang, M. K. Jangid, J. Niu, C. Wang, and Y. Zhang. [A Formal Analysis of Agent Payment Protocols](#). arXiv:2609.00060, August 30, 2026. Preprint. Tamarin analysis of x402, MPP, ACP, and AP2.
- [31] U. Uchibeke. [Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents](#). arXiv:2603.20953, March 21, 2026. Preprint.
- [32] Z. Zhang and X. Zhang. [Are You Still the Agent I Authorized? Earned Authority under a Fixed Ceiling for Evolving Agents](#). arXiv:2607.23586, July 26, 2026. Preprint.
- [33] A. Aviv, P. A. Gandh, R. Bitton, and A. Shabtai. [Beyond the Mandate: A Systematic Security Analysis of the Agent Payments Protocol \(AP2\)](#). arXiv:2608.23858, August 24, 2026. Preprint.
- [34] A. Allouah, O. Besbes, J. D. Figueroa, Y. Kanoria, and A. Kumar. [What Is Your AI Agent Buying? Evaluation, Biases, Model Dependence, and Emerging Implications for Agentic E-Commerce](#). arXiv:2508.02630, August 4, 2025. Preprint.
- [35] American Express. [American Express Debuts Agentic Commerce Experiences \(ACE\) Developer Kit and Announces Industry-First Protection for Registered Agent Purchases](#). Company newsroom, April 2026 (dated April 14, 2026 in contemporaneous [Fortune](#) coverage). Quoted directly; the protection language is forward-looking and conditional. Secondary framing in [The Financial Brand](#), May 2026, unbylined.
- [36] A. Asgaonkar and B. Krishnamachari. [Solving the Buyer and Seller’s Dilemma: A Dual-Deposit Escrow Smart Contract for Provably Cheat-Proof Delivery and Payment for a Digital Good without a Trusted Mediator](#). arXiv:1806.08379, 2018; published at IEEE ICBC. Cited for its explicit assumption that the good is hash-verifiable.
- [37] E. Debenedetti et al. [AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents](#). NeurIPS 2024 (Datasets and Benchmarks). Peer-reviewed.

- [38] C. Busch. German Law Journal, 2025. Peer-reviewed treatment of AI agents in consumer contract law. Full citation to be confirmed before circulation.
- [39] N. Kolt. Governing AI Agents. Notre Dame Law Review, forthcoming. Peer-reviewed.
- [40] E. Debenedetti et al. [Defeating Prompt Injections by Design](#) (CaMeL). arXiv:2503.18813, 2025. Preprint.
- [41] Uniform Law Commission. [Uniform Electronic Transactions Act](#) (1999), §10, “Effect of change or error.”
- [42] Natural. [Create payment](#). API reference; request fields, `tags` constraints, Payment status enum. Accessed September 19, 2026.
- [43] Natural. [Get agent](#). API reference; Agent object and `limits`. Accessed September 19, 2026.
- [44] Natural. [Event types](#). Webhook event catalog; `payment.*`, `mandate.*`, `chargeback.*` payloads quoted in Section 8.1. Accessed September 19, 2026.

Appendix: prototype evidence boundary

The local verification used the Scrip repository (project directory: `spending`), branch `main`.

Commits designating the tested code:

```
97136f0 Add SPEC.md and implementation brief
200ff6b Add the purchase mission slice
5c07c79 Add agent identity with version-bound authority and close four proven exploits
```

The demonstration (`demo/deterministic-authorization.ts`, `tests/demo-deterministic.test.ts`) and this revision of the paper are committed immediately after `5c07c79`; the repository history names that commit.

Commands executed on September 19, 2026, all exit status 0:

```
npm run build
npx tsc --noEmit
npm test
npm run demo:deterministic
```

Result: 253 passed, 8 skipped, across 22 passing test files and one skipped file. Toolchain: Node 24.11.0, pandoc 3.9, Tectonic 0.17.

Source paths are relative to the repository root.

- Agent identity, manifest classification, attestation levels: `src/missions/agent-identity.ts`.
- Authentication and authority: `SQLiteAgentRegistry` in `src/missions/agent-registry.ts`, especially `registerVersion()`, `authenticate()`, `verifyCredential()`, and `authorize()`.
- Orchestration: `PurchaseMissionService` in `src/missions/purchase-mission-service.ts`, especially `authenticate()`, `checkAuthority()`, `execute()`, and `reconcile()`.
- Assessment and constraint evaluation: `preflight()` and `assessOutcome()` in `src/missions/outcome-assessor.ts`.
- Demonstration: `runDemo()` in `demo/deterministic-authorization.ts`.
- Adversarial and identity fixtures: `tests/agent-identity.test.ts` (26 tests, including the four attacks in Section 9.2).

- Purchase fixtures: **tests/purchase-mission-service.test.ts** (50 tests).

SHA-256 digests of the five inspected files at the time of this revision, split across two lines for print; concatenate without whitespace.

```
agent-identity.ts  
fffaa3cba434782ed1b25bc382353ffa  
d2ae666c0dd0f9716f7f9e294aaf712f
```

```
agent-registry.ts  
05ccc2b61a001e587aa26199e7a41731  
98e23da532d20f34184dbba72fb51552
```

```
purchase-mission-service.ts  
791405f8d8fa572b1463022012ad1189  
d865934f0cffc8e790043e9323629f54
```

```
outcome-assessor.ts  
9217343b5538517b74495ac5c8542222  
35f1b70fb7c5028643530c5b80c78269
```

```
deterministic-authorization.ts  
492c00fb57fa2f43e03fba3ddba84419  
fbe214bb2f369e5f4844c78d8d21352f
```